Isolated at the Route, Leaking Below: Detecting Derived-State Tenant Isolation Failures in Multi-Tenant RAG Applications

Lisheng Ni
Independent Researcher
Email: 1175235190@qq.com

***Abstract*—Retrieval-augmented generation (RAG) applications increasingly serve many users from one shared instance over shared infrastructure: vector collections, lexical indexes, generation and question caches, and background indexing workers. Access control is enforced at the API *route*, but the answer a tenant receives is assembled from *derived state* that the route never inspects. We show that this structural gap produces cross-tenant leaks that route-level authorization testing—including Broken-Object-Level-Authorization (BOLA) scanning—is blind to, and we give a black-box method to find them before launch. *Canary-echo differential testing* plants owner-identifying canary tokens in each tenant's documents, replaces the chat and embedding models with a deterministic, OpenAI-compatible *echo* service (no GPU, no API key), and decides isolation with four content-based oracles over user output, LLM prompts, persisted artifacts, and route-level denials. We organize the failures the method targets into a taxonomy of six derived-state leak classes (D1–D6). We evaluate along three axes. *(i) Cross-application:* across six open-source multi-tenant RAG systems the method surfaces two independent real cross-tenant leaks beneath correctly authorized routes—a version-differential derived-layer leak in Open WebUI and a dynamic reproduction of an unfixed sharing-authorization CVE in Khoj—alongside correctly-isolating systems, with no false alarms. *(ii) Historical:* across ten public isolation CVEs in ten RAG applications, every advisory is a route-level (D5) authorization bug; the derived-layer classes are absent from the public record. *(iii) Effectiveness:* on seeded faults, the method detects all five injected classes with no false positives, versus 20% for a route-level IDOR scanner and 80% (with a false positive) for a static key-analysis baseline. The tool, scenarios, adapters, and data are released.**

***Index Terms*—software reliability, security testing, tenant isolation, retrieval-augmented generation, broken object-level authorization, metamorphic testing, differential testing.**

I. Introduction

Retrieval-augmented generation (RAG) [1] has become the default architecture for knowledge assistants. A user's question is embedded [2], [3], matched against an index of document chunks—by dense nearest-neighbor search [4], [5], by lexical BM25 [6], or by a fusion of several signals [7]—and the retrieved passages are placed in the prompt of a large language model that composes the answer [8]. In education, enterprises, and software-as-a-service products, these assistants are deployed *multi-tenant*: a single instance serves many users, each with private documents, over shared storage and shared retrieval indexes.

Isolation between tenants is, before it is a security issue, a software *reliability* problem— and a particularly insidious one. A system that silently returns one customer's confidential document to another has failed in service, yet every request returns a 200, no exception is thrown, and the latency is nominal: the failure produces no observable *event*. It is therefore invisible to functional tests (each tenant's own queries answer correctly), invisible to health and performance monitoring (nothing crashes or slows), and invisible even to software-reliability growth models, which estimate failure intensity and MTBF from an observed failure process and cannot count a failure that emits no signal [9]. The defect surfaces only as a correctness property *across* tenants—a relation between two executions that conventional per-request testing does not state. This is the defining shape of a *silent* reliability failure (we develop the framing in Section VIII-A), and it is the class a dependable multi-tenant service most needs to exclude, because the damage is irreversible once an answer reaches a user. The OWASP Top 10 for LLM applications names cross-tenant leakage a first-class risk [10] and the OWASP API Security Top 10 ranks broken object-level authorization (BOLA) the top API risk [11]; yet, as we show, the public record contains *no* reported instance of the derived-state failure mode we study, which suggests it is under-tested rather than absent.

Most security research on RAG studies attacks on the *model and its data*: extraction of training or context data [12], [13], indirect prompt injection through retrieved content [14], and knowledge-base poisoning [15]. We study a different and, we argue, more mundane but more pervasive failure: a tenant who is *correctly denied* a document through the API still receives its *content* through retrieval. This is not a model attack; it is an engineering failure of isolation in the data path, and it is a reliability defect that should be caught before release.

A. Why it happens

A RAG request is authorized at the *route*. The question "may user u read file f?" is checked by the object-access handler, and if f is not u's, the handler returns a denial. But the *answer* is not assembled from that handler; it is assembled from *derived state*—a vector namespace, a lexical index, a generation or question cache, an index cached inside a background worker—whose keys are chosen for retrieval quality, not for ownership. When that state is keyed by a resource identifier or a content

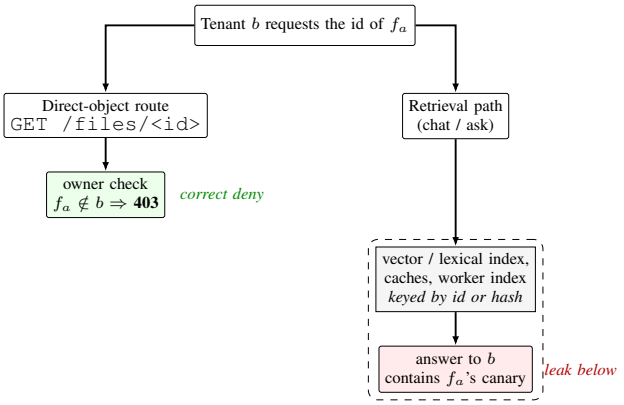


Fig. 1. The same stored content is reachable by two paths. The direct-object route consults the owner check and correctly denies a foreign id; the retrieval path reads from derived state keyed by id or content hash and never consults it. Route-level authorization testing observes only the left branch.

hash rather than by owner, a request that is correctly authorized at the route can still pull another tenant's content from beneath it. Figure 1 makes the gap concrete: the direct-object route and the retrieval path reach the same stored content through *different* code, and only the former consults the owner check. The cross-tenant leakage literature in cloud computing established long ago that *sharing infrastructure creates channels the application layer does not see* [16], [17]; RAG re-introduces the same hazard *inside* a single process, in its retrieval indexes and caches.

B. Why route-level testing misses it

The mature line of work on authorization testing for web and REST APIs—BOLA/IDOR and Broken-Function-Level-Authorization detection via fuzzing and specification analysis [18], [19], [20], [21]—operates at exactly the layer that is implemented *correctly* in these applications. Such tools infer a violation from an HTTP status code: a request that should be 403 but returns 200. But a derived-layer leak produces a *correct* 403 on the direct object route and leaks through a *different* path—retrieval. A status-code oracle at the route cannot see it. We confirm this empirically in two independent ways: the public CVE record for RAG applications is *entirely* route-level (Section VI), and a route-level IDOR scanner detects only one of five injected leak classes (Section VII).

C. Approach and contributions

We make the derived layer observable without a GPU or a paid model. Each tenant's documents carry *content canaries*: dictionary-compound tokens chosen to survive lexical tokenization as a single term and to pass through an LLM verbatim; every question additionally carries a per-asker *query canary*, so that each prompt reaching the model can be attributed to the tenant that triggered it. The application's chat *and* embedding endpoints are pointed at an *echo* service—a small OpenAI-compatible server that echoes the canaries it is given, returns deterministic embeddings so the application's real dense and lexical retrieval still runs, and logs every prompt with its attributed actor. A leak is then one predicate: a canary owned by tenant a observed in a context attributed to tenant $b \neq a$. That predicate is a metamorphic, cross-tenant *differential* relation that canaries make concrete and that sidesteps the generative oracle problem (developed in Section II). Our contributions are:

1) **Method.** CANARY-ECHO differential testing: the first black-box, GPU-free method that makes the derived layer observable and decides tenant isolation as a two-tenant metamorphic relation (Section III). A load-bearing design finding is that the route-level denial oracle must be *content*-based, not status-code based (Section III-D).
2) **Taxonomy.** Six derived-state leak classes (D1–D6) that name the *keying mistake* rather than the symptom, with minimal reproducible examples (Section IV).
3) **Cross-application study (RQ1).** Six open-source multi-tenant RAG applications; two independent real-world cross-tenant leaks below correctly authorized routes—a version-differential derived-layer leak in Open WebUI and a dynamic reproduction of an unfixed CVE in Khoj—two clean dynamic negatives, and two opposite source-level verdicts, with no false alarms (Section V).
4) **Historical study (RQ2).** Ten public isolation CVEs across ten RAG applications, all route-level (D5), with independent-coder reliability (Section VI).
5) **Effectiveness (RQ3).** On seeded faults over 1000 randomized trials per class, CANARY-ECHO reaches 99.3% aggregate recall (Wilson 95% CI $[99.0\%, 99.5\%]$) with zero false positives, versus 20.0% for route-level IDOR scanning and 80.0% (with a 100% guarded-store false-positive rate) for static key analysis (Section VII).

All artifacts—the tool, the scenario suite and adapters, the seeded application, the CVE corpus and the κ coding kit—are released at https://github.com/your-github-id/rag-trust-lab, together with a reproducibility guide that maps every quantitative claim in this paper to the exact command that regenerates it; the deterministic studies (RQ2's κ and all of RQ3, including the 1000-trial robustness run) are bit-for-bit reproducible.

II. Background and Threat Model

A. The derived state of a RAG pipeline

A multi-tenant RAG instance maintains, per tenant or shared across tenants, several pieces of *derived* state computed from uploaded documents:

- a **dense vector index** (an approximate-nearest-neighbor structure [4], [5]) whose entries map an embedding to a chunk;
- a **lexical index** (e.g. BM25 [6]) over the same chunks, often fused with the dense scores [7];
- **caches**: a generation cache (answers, summaries), a question or quiz cache, or a semantic cache keyed by the normalized query or by a content hash;
- **in-memory indexes inside worker processes** that serve retrieval and may outlive the database rows they were built from;

- **client-side state** (browser storage) holding chat history or selections.

Each is keyed for retrieval efficiency, not for ownership. Correct isolation requires that *every* such key also scope by owner (or by an owner-bound namespace), and that deletions propagate to all of them synchronously. The route handler that authorizes `GET /files/<id>` has no visibility into any of these keys, and in general cannot: the indexes are built asynchronously, by a different component, often in a different process, and are queried by *similarity*, not by identifier. This is the structural reason the two paths in Figure 1 diverge.

B. Isolation as a cross-tenant correctness property

A convenient way to state the property under test is as a relation over pairs of tenants rather than over single requests. Let $C(t)$ be the set of content canaries owned by tenant t, and let $\mathsf{obs}(t)$ be the set of canaries observed in any context attributable to t—its answers, the prompts its requests induce, and the artifacts its actions persist. Isolation holds iff, for every pair $a \neq b$ that are not intentionally sharing,

$$\mathsf{obs}(b) \cap C(a) = \varnothing.$$

This is a *safety* property over a two-tenant composition, and it is *metamorphic*: it constrains the relationship between two executions (tenant a's uploads and tenant b's queries) rather than the absolute output of either [22], [23]. Framing isolation this way is what lets us test a generative system whose single-request output has no decidable oracle [24]: we never ask whether b's answer is *correct*, only whether it contains a token it provably should not.

C. Tenants and adversary

A *tenant* is an authenticated user who uploads private documents and asks questions answered over them; tenants may be grouped into workspaces, agents, or shared "brains," and isolation is expected between tenants that are not intentionally sharing. Our *adversary* is an authenticated tenant with an ordinary account and no server access, who may (a) enumerate or reuse resource identifiers, (b) place another tenant's resource id on any API that accepts one—*including the retrieval path*, not only the direct-object route—and (c) use otherwise-normal sharing or fork features in unintended combinations. We also consider *benign* mis-isolation with no adversary at all: identifier reuse after a delete, stale state in a second worker, or shared-device client storage, each of which can surface one tenant's content to another with no malice. Out of scope are prompt injection [14], model or training-data extraction [12], knowledge poisoning [15], and micro-architectural side channels on the serving infrastructure [17]. We target *isolation of tenant content across the derived state of the RAG pipeline*.

III. Canary-Echo Differential Testing

A. Overview

The method instantiates the relation of Section II as an executable differential test. At a high level (Algorithm 1): seed each tenant's documents with owner-tagged canaries; drive the application through a fixed battery of two-tenant scenarios with its models pointed at the echo service; collect every observation point (answers, prompts, artifacts, route responses); and flag any observation attributed to b that contains a canary owned by $a \neq b$. The procedure is black-box—it needs only the application's ordinary HTTP surface and a configurable model endpoint—and deterministic, so a flag reproduces.

Algorithm 1: Canary-echo differential isolation test

Input: application A; tenants $\{a, b, \dots\}$; scenario battery S; canary registry R
Output: set of leaks $(\text{owner}, \text{observer}, \text{oracle}, \text{scenario})$
point A's chat and embedding endpoints at the echo service E
foreach *tenant* t **do**
  create a *regular* (non-admin) account for t
  foreach *document* d *uploaded by* t **do**
    seed d with a fresh content canary c; $R[c] \leftarrow t$
leaks $\leftarrow \varnothing$
foreach *scenario* $s \in S$ **do**
  execute s; each query by t carries t's query canary
  foreach *observation* o *attributed to observer* t_o **do**
    foreach *canary* $c \in o$ *with* $R[c] \neq t_o$ **do**
      leaks $\leftarrow$ leaks $\cup \{(R[c],\ t_o,\ \text{oracle}(o),\ s)\}$
return leaks

B. Canaries

Each document a tenant uploads is seeded with a *content canary*: a dictionary-compound token such as `OSCARURIAL85`. The design has three requirements. (i) It must survive *lexical* tokenization as a single term, so that BM25 [6] indexes it verbatim—hence a compound of real dictionary stems rather than random characters, which tokenizers split or drop and which would therefore become unsearchable through the lexical path. (ii) It must pass through an LLM *verbatim*, so that it is recoverable from a generated answer without paraphrase loss. (iii) It must be recognizable by a fixed regular expression so that detection needs no per-document state and no natural-language matching. A registry maps each canary to its owner and its *kind* (document body, title, metadata), which lets an oracle report not only *that* a leak occurred but *which* derived field carried it. Every question additionally carries a per-asker *query canary*; because the asker's query canary accompanies the retrieved context into the prompt, any prompt reaching the model can be attributed to the single tenant whose query canary it contains. This attribution is what lets us name the actor of a background job or a shared-cache generation, where the HTTP request that ultimately surfaces the content may belong to no one in particular.

C. The echo service

The application's chat *and* embedding endpoints are configured to a small OpenAI-compatible *echo* service. For `/v1/chat/completions` it returns the canaries present in the prompt (streaming or non-streaming, matching the client's request), so that cross-tenant context surfaces in the UI or API exactly as a real model would surface it. For `/v1/embeddings` it returns *deterministic hashing embeddings*, so that the

```
POST /v1/chat/completions
{ "messages": [
   {"role":"system","content":"...context: OSCARURIAL85
        ..."},
   {"role":"user","content":"...query canary QZB7TENANTB
        ..."} ] }
--> 200
{ "choices": [{"message":{"role":"assistant",
    "content":"echo: OSCARURIAL85 QZB7TENANTB"}}] }
log: actor=tenant_b canaries={OSCARURIAL85(owner=a),
     QZB7TENANTB(owner=b)}
    => O-prompt LEAK: a-canary in b-attributed prompt
```

Listing 1. An echo exchange. The chat request carries tenant b's query canary and the retrieved context; the echo service returns the canaries verbatim and logs the prompt under b. If the returned set contains a canary owned by a, O-prompt fires.

application's real dense vector search runs and is reproducible and the same canary is reachable by both the dense and the lexical path. It logs every prompt with its attributed actor. Listing 1 shows the shape of an exchange. The service needs no GPU, no model weights, and no API key; any application that can point its models at a custom OpenAI-compatible base URL—which is the large majority of self-hosted RAG systems—can be tested. Deterministic embeddings deliberately trade retrieval *quality* for *reproducibility*: we are testing which store serves which tenant, not ranking accuracy, and the canary predicate is independent of embedding fidelity. A real model would add nondeterminism and cost without changing any oracle's verdict, because the verdict is a substring test, not a judgment of answer quality.

D. Oracles

A leak is a canary owned by a observed in a context attributed to $b \neq a$. Four oracles read different observation points, which matters because an application may suppress a leak at one point while exposing it at another:

- **O-out:** a foreign canary in a user-visible answer—the end-to-end symptom;
- **O-prompt:** a foreign canary in an LLM prompt attributed (by query canary) to another actor—the derived-layer signal, visible even when the UI suppresses the content, and the earliest point at which the leak is observable;
- **O-art:** a foreign canary in a persisted derived artifact (a cache file, an index, an export) keyed to another owner;
- **O-authz:** a route-level request for a foreign resource that nonetheless returns that resource's content.

a) The route-level oracle must be content-based: A natural implementation of O-authz is a status-code rule: the attacker's request to a foreign resource "passes" iff it is denied with $401/403/404$, as BOLA fuzzers assume [21], [18]. We found this rule unsound in both directions. Applications signal denial with *different* statuses—$401/403/404$, but also 400, and even 200 with a null body—so a status rule *false-positives* (a 200-with-null denial read as a leak) and, more dangerously, *false-negatives* (a 200 that *does* return foreign content read as success because its status looked benign). We therefore decide O-authz by the *same* canary predicate as the other oracles: the attacker passes iff no foreign canary is returned, and the HTTP status is recorded only for transparency (we tag whether the denial was a "hard" status or a 200-null). On one correctly-isolating application (Section V) this change removed two spurious failures that a status rule reported, and it is the single most important robustness decision in the oracle design.

b) Soundness of the leak signal: A false positive would require a canary owned by a to appear in b's context *without* any cross-tenant flow. Because content canaries are high-entropy tokens minted per document and never reused, and because the matching regular expression admits no natural-language near-misses, the only way a's canary reaches b is for some component to have served a's derived state to b—which is precisely the failure under test. The signal is therefore dispositive rather than heuristic: a positive is a proof of leakage, not an anomaly score, and the method reports no severity ranking or confidence because none is needed. The corresponding risk is a false *negative*—a leak the scenario battery never triggered—which is why S1 (own-document retrieval) is mandatory: it certifies that retrieval ran and carried a canary at all, turning an untested path into a visible gap rather than a silent pass.

E. Scenario suite

An application-agnostic suite drives one application through a fixed set of scenarios that systematically stress each keying mistake:

- **S1** own-document retrieval—a sanity check that retrieval actually runs and carries the canary into the prompt; without it a "clean" result could be a false negative caused by a mis-wired adapter rather than by sound isolation;
- **S2** explicit cross-ask—a tenant asks for another's content library-wide;
- **S3** byte-identical copy for a second tenant—exercises shared caches and content-addressed sharing (D4);
- **S4** delete and re-upload—exercises identifier reuse and stale derived state (D2);
- **S5** foreign resource ids on direct-object routes—the BOLA/IDOR surface (D5);
- **S6** derived-artifact scan—inspects persisted caches and indexes (D3);
- **S7** *derived-layer reference*—the attacker references a victim's raw resource id *in the retrieval path*, bypassing the direct-object route; this is the class the route-level tests structurally miss (D1);
- **S8** delete-residue—after the victim deletes a resource, the attacker references it through the derived store (D2/D3);
- **P** a prompt audit requiring that every prompt attributed to a tenant contains only that tenant's canaries—the suite-wide invariant behind O-prompt.

F. Adapters and administrator bootstrap

A per-application *adapter* (typically 80–180 lines) knows only how to create a regular user, upload a document, ask a question restricted to given resources, delete a resource, substitute foreign resource ids on direct-object routes, and—optionally—reference a raw resource id in the retrieval path.

Everything else is application-agnostic. Two execution details are load-bearing. First, test subjects must be *regular* users: an adapter that makes the first account an administrator by construction would count that account's by-design cross-tenant access as a leak, so the suite bootstraps a separate non-subject administrator and creates all subjects as regular users. Second, scenarios that depend on stale worker state (S4/S8, class D2) require retrieval to run across more than one worker process over a shared store; a single-worker run hides the staleness and yields a false negative, so the adapter declares the application's worker topology and the harness provisions accordingly.

IV. A Taxonomy of Derived-State Leaks

We name each class by the *keying mistake*, which is what generalizes across applications; the symptom (which canary surfaces where) varies, the mistake does not. The first four classes (D1–D4) live in server-side derived state, D5 at the route, and D6 on the client.

- **D1** *Owner-less derived index.* A vector or lexical index keyed by resource id alone (`material_{id}`, `bm25_{id}`), reachable through retrieval even when the direct-object API is correct. The route denies; the index does not know there was a question to ask.
- **D2** *Identifier reuse with stale derived state.* Resource ids recycled—through delete-and-restore, auto-increment reset, or backup import—while a stale derived index (for example, inside a second worker that has not reloaded) survives and serves the old content to whoever now holds the id.
- **D3** *Release-artifact / orphan cache contamination.* Persisted caches or indexes shipped inside the release build, or left as orphans after deletes, validated only by a weak fingerprint such as a chunk count, so that a new tenant's resource is silently matched to a stale cache entry.
- **D4** *Content-addressed cross-tenant sharing.* A cache keyed by content hash and shared across tenants; beyond leaking content, the key doubles as an existence oracle—"has anyone on this instance uploaded this exact file?"—which is itself a confidentiality breach.
- **D5** *Client-asserted identity or scope.* Identity or scope taken from the request body, or ownership checked only *after* the action has run; the route-level class that BOLA/IDOR testing targets and that fills the public CVE record.
- **D6** *Client-side persistence across accounts.* Browser storage keyed globally, so chat history or selections survive logout or an account switch on a shared device.

Our three evaluations show, respectively, that real *public* isolation bugs cluster in D5 (Section VI); that real D1–D2 leaks nonetheless occur in deployed applications and are found by our method (Section V); and that the method detects all server-side classes where route-level and static baselines do not (Section VII). Table I gives a minimal reproducible example of each server-side class on a controlled application, each leaking before and clean after the corresponding fix—evidence that the classes are not merely conceptual but each correspond to a concrete, independently repairable defect.

TABLE I
Seeded minimal reproducible examples: each class leaks before its fix and is clean after.

Class	Keying mistake (seeded)	Vuln.	Fixed
D1	owner-less derived index (id only)	leak	clean
D2	identifier reuse, stale worker index	leak	clean
D3	shipped / orphan cache, weak fingerprint	leak	clean
D4	content-hash cache shared across tenants	leak	clean

```
# vulnerable: owner-less derived index (D1)
chunks = index.search(embedding, where={"source_id": sid})

# fixed: owner-scoped retrieval filter
chunks = index.search(embedding,
   where={"source_id": sid, "owner": current_user.id})
```

Listing 2. D1 in one line. The retrieval query selects chunks by resource id alone (top); the fix (bottom) scopes the same query by owner. The direct-object route is unchanged and may be correct in both cases.

The D1 mistake is instructive because it is a one-token omission that no route test can observe. Listing 2 contrasts a vulnerable owner-less retrieval query with its owner-scoped fix: the direct-object route above it may be flawless, yet the query beneath it reaches every tenant's chunks. The repair is to add the owner predicate to the *retrieval* filter, not only to the route—and the only way to know it was missing, short of reading every query, is to observe that a foreign canary came back.

V. RQ1: Cross-Application Study

RQ1: Does the method find real derived-state isolation failures in deployed open-source RAG applications, without false alarms on correctly isolating ones?

A. Setup

We selected six open-source multi-tenant RAG applications spanning distinct architectures and vector stores, each self-deployed in isolation on hardware we control, with its chat and embedding models pointed at the echo service. Four expose an HTTP API sufficient to drive the suite dynamically; two do not—one is a Gradio-only application with no stable REST surface, and one is a four-container RAG stack whose memory footprint exceeded our isolated test host—and for those two we confirm the isolation property at the source level against the exact released revision, reading the ownership predicate on every path that reaches the derived store. Every dynamic finding is accompanied by a positive control (a configuration or version known to leak) and a negative control (one known to be sound), so that a "leak" verdict is never an artifact of a mis-wired adapter and a "clean" verdict is never the silent failure of a test that never exercised retrieval. Table II lists the six applications and the axes along which

TABLE II
THE SIX APPLICATIONS UNDER STUDY AND THE AXES ALONG WHICH THEY DIFFER.

Application	Interface	Vector store	Tenancy unit
Open WebUI	REST	shared collection	user
Khoj	REST (Django)	pgvector	user / agent
AnythingLLM	REST	per-workspace ns	workspace
LangFlow	REST	per-user store	user
Kotaemon	Gradio only	local index	user
LibreChat	REST + RAG svc	pgvector (svc)	user

they differ—interface, vector store, and the unit over which tenancy is defined—and Table III summarizes the results. The spread matters: a method that generalizes must work across a shared single collection (Open WebUI), per-namespace stores (AnythingLLM), a relational pgvector backend (Khoj, LibreChat), and a tenancy unit that is sometimes a user, sometimes a workspace, and sometimes an agent.

B. Flagship case: Open WebUI (version-differential)

On a pinned vulnerable image (0.8.11) the suite's checks fail at exactly two points. In **S7**, a regular user supplies a *foreign* `file_id` in the chat request's file-reference list; the retrieval path loads that file's content and places it into the attacker's own prompt, where the victim's content canary appears (O-prompt and O-out both fire). In **S8**, after the victim *deletes* the document, its entries in the shared vector collection are not purged and remain retrievable by another user. Crucially, the *route-level* checks (S5) *pass*: the same user calling `GET /files/<id>` on the victim's id is correctly denied with 404. This is the pattern the paper is named for—isolated at the route, leaking below. A pinned *fixed* image (0.12.5) passes all checks, a clean negative control, and a role-aware positive control reproduces the application's public route-level CVE so that we know the harness can see a leak when one exists.

A version bisection localizes the derived-layer fix to the interval $(0.8.11,\ 0.10.2]$, consistent with the vendor's stated fix release. The intermediate 0.9.5 is *not* adjudicable by this oracle because its file-reference endpoint rejects *any* file reference—own or foreign—with a server error, which an own-file control (S1) confirms is an API change rather than an ownership check. Reporting that honestly, rather than scoring 0.9.5 as "fixed," is exactly the kind of control the canary oracle makes cheap: because S1 distinguishes "retrieval is off" from "retrieval is isolated," we avoid crediting a sound verdict to what is really a broken endpoint.

C. Second positive: Khoj (a live, unfixed CVE)

Using only documented REST endpoints, tenant B forks a conversation that tenant A shared publicly and thereby obtains a conversation *bound to A's private agent*: the fork copies the agent with no access check. We confirm the leak with the application's own code: Khoj's authorization predicate `ais_agent_accessible(agent, B)` returns `False`—the check the fork-then-chat path omits—while the agent's knowledge base holds A's private canary, which the chat retrieval selects with `user=None` and `owner_filter=Q(agent)`. A control in which B chats without the forked agent sees nothing of A's, so the leak is specific to the asymmetric share/fork→agent-knowledge path, not to personal documents. We note a method limitation here: the full chat-level answer could not be elicited with the deterministic echo backend, because Khoj's pipeline uses the model itself to decide *whether and what* to retrieve, and the echo service does not satisfy those intermediate structured decisions; we therefore confirm at the retrieval/authorization layer using the application's real code rather than overclaiming a UI-level reproduction. This case shows both the reach of the method (it localized a live, unfixed cross-tenant leak) and an honest boundary of the echo technique (Section VIII-B).

D. Clean negatives and precision

AnythingLLM and LangFlow isolate correctly and produce *no* false alarms. AnythingLLM is the case that motivated the content-based O-authz oracle (Section III-D): it denies cross-workspace access with 200-plus-null and 400 responses that a status-code rule flags as leaks even though no canary is ever returned. With the content-based oracle, all of its checks pass, and the two spurious failures disappear. That a precision fix on one application is driven by an observation on another is itself an argument for a uniform, content-based oracle rather than a per-application status allowlist.

E. Source-level confirmations

Kotaemon, a Gradio-only application without a REST surface, is confirmed *positive* by source against its released revision: four conversation handlers fetch a conversation by id with no owner predicate, in contrast to a sibling history loader that filters by owner—an unfixed CVE—and the retrieval query selects chunks by `source_id` alone, a latent owner-less-index (D1) surface. LibreChat, a four-container RAG stack, is confirmed *negative*: a message-referenced file is admitted only if present in an owner-scoped map, the agents path loads attachments with a user-scoped query, and the retrieval service additionally filters by a per-user token, so a foreign `file_id` is dropped before it reaches retrieval; its historical credential-IDOR is fixed in the current release. The two opposite source-level verdicts show the taxonomy discriminates in both directions, not only toward "leak," and that the method's negative verdicts are falsifiable claims about specific predicates in specific code, not an absence of evidence.

F. Answer to RQ1

The method found two independent real-world cross-tenant leaks beneath correctly authorized routes (one a version-differential derived-layer leak, one a live unfixed CVE), returned clean negatives on two correctly-isolating applications with no false alarms, and produced two opposite verdicts at the source level. In every positive case the route-level checks passed while the derived-layer checks failed—the precise gap route-level testing cannot observe.

TABLE III
RQ1 CROSS-APPLICATION RESULTS. "DYN." = DRIVEN DYNAMICALLY OVER HTTP; "SRC" = SOURCE-LEVEL CONFIRMATION AGAINST THE RELEASED REVISION.

Application	Mode	Layer	Verdict	Finding
Open WebUI	Dyn.	route + derived	**leak (D1, D2)**	A regular user reads a foreign file's content via the chat file-reference path (S7) and via delete-residue in the shared vector store (S8), while the direct file API returns 404 for the same id (S5 passes). Present in 0.8.11, absent from 0.10.2 onward.
Khoj	Dyn.	route + derived	**leak (D5→KB)**	CVE-2026-13508 (fix PR unmerged): a user forks a publicly shared conversation and obtains one bound to the author's *private* agent; the app's own agent-access predicate returns false for the forker, yet the agent's knowledge base holds the author's canary, which the chat retrieval selects via an agent-scoped filter with no user bound.
AnythingLLM	Dyn.	route + derived	clean	Per-workspace vector namespace plus membership-checked chat; cross-workspace access is denied (via 200-null / 400 / 404) with no content leak.
LangFlow	Dyn.	route (object)	clean	Cross-user object access returns 404; no content leak on any path the adapter exercises.
Kotaemon	Src	route + derived	**leak (D5, unfixed)**	CVE-2026-82281 (unfixed at HEAD): four conversation handlers fetch by id with no owner predicate (the sibling history loader has one); retrieval additionally selects by `source_id` alone, a latent D1 surface.
LibreChat	Src	route + derived	clean	Message-referenced files are admitted only from an owner-scoped map ({`file_id, user`}) and the RAG service filters by a per-user token; a foreign `file_id` is dropped before retrieval. Its historical credential-IDOR CVE is fixed.

TABLE IV
RQ2: TEN PUBLIC ISOLATION CVES; EVERY ONE IS A ROUTE-LEVEL (D5) CLASS. "—" = NO RELEASED FIX AT THE TIME OF WRITING.

Application	CVE	CWE	Fixed
Open WebUI	2026-45402	639	0.9.5
AnythingLLM	2026-55611	639	1.14.1
Dify	2026-34082	284/863	1.13.1
RAGFlow	2026-51892	284	—
LibreChat	2026-31942	862	0.8.3-rc1
Kotaemon	2026-82281	639	—
Onyx	2024-7767	862	—
FastGPT	2026-61644	863	4.15.0-beta5
Khoj	2026-13508	285/863	— (PR open)
Quivr	2026-82280	639	—

VI. RQ2: WHAT THE PUBLIC RECORD CONTAINS

RQ2: Are derived-state isolation bugs represented in the public vulnerability record for RAG applications, and how reliably can isolation bugs be classified into D1–D6?

A. Corpus and method

We assembled, from OSV, the NVD, and GitHub Security Advisories, one public isolation CVE for each of ten open-source RAG applications, verifying each against its authoritative advisory and, where a fix exists, against the upstream repository (tag, merge-base, or fix pull request). During verification we corrected a subtlety that matters for honesty: for three applications the identifier recorded by aggregators as the "fixed" commit is a *release* commit (a version bump) rather than the granular security patch, so we report the fixed *version* and do not attribute a specific line-level fix we did not confirm. We then classified each CVE into D1–D6. Table IV lists the corpus.

B. Finding: the public record is entirely route-level

All ten advisories are route-level object- or function-authorization bugs (D5): CWE-639 (authorization bypass through a user-controlled key, $\times 4$), CWE-862 (missing authorization, $\times 2$), CWE-863 (incorrect authorization), CWE-284, CWE-285, and combinations. *Not one* is a derived-layer (D1–D4) leak. We read this not as a sampling artifact but as the result: public advisories record authorization bugs that can be demonstrated in a single route call and mapped cleanly to a CWE, which is precisely where route-level BOLA testing [21], [19], [20] and the broader REST-API-testing literature [25], [26], [27] are strong. Derived-state leaks keyed by identifier, content hash, or stale index are systematically *absent*. The one public case that touches a derived index is an id-manipulation that *poisons* another tenant's index—a write reached through a route-level flaw, aligned with knowledge-poisoning attacks [15], not the cross-tenant *read* our method surfaces. A second observation on the disclosure landscape: only five of the ten carry a released fix; the others are unpatched, carry an open fix pull request, or had the vulnerable code removed only incidentally in a later rewrite—so these are not merely historical curiosities but a live state of the ecosystem.

C. Classification reliability

Reliability of a manual taxonomy is standard to report with an inter-coder agreement coefficient [28], [29]. One author and an *independent LLM annotator*—given only the codebook and neutral one-line descriptions, with no project context and no sight of the author's labels—classified a mixed set of thirteen cases (the ten route-level CVEs plus three derived-layer cases: our two Open WebUI findings and the public index-poisoning case). A single-category set would make κ degenerate (observed and expected agreement both 1), so the derived-layer cases are included deliberately to give the scale range. Agreement on the

TABLE V
RQ3 CANONICAL SINGLE-FAULT RESULT: DETECTION OF FIVE SEEDED CLASSES AND FALSE POSITIVES (FP).

Detector	D1	D2	D3	D4	D5	Recall	FP
CANARY-ECHO (ours)	✓	✓	✓	✓	✓	100%	0
IDOR scanner					✓	20%	0
Static keying	✓	✓	✓	✓		80%	1

ten route-level CVEs was 100% (both assigned D5 to all ten); over the full set Cohen's $\kappa = 0.46$ (moderate on the Landis–Koch scale [29]). All disagreement fell on the two cases at the route-vs-derived boundary, where the annotator read an id-manipulation framing as route-level and the author read the "direct API denies, retrieval leaks" framing as derived-layer—the very subtlety that makes these leaks easy to overlook, and that motivates a testing method rather than a classification exercise. We disclose that the second annotator is an LLM, not a human expert; human replication is a stronger follow-up, and the blind coding materials are released for it.

VII. RQ3: EFFECTIVENESS ON SEEDED FAULTS

RQ3: On faults of known class, how does the method's detection compare to route-level IDOR scanning and to static key analysis?

A. Setup

Seeding faults of a known class to measure a detector's effectiveness is the standard fault-injection methodology used throughout mutation and reliability testing [9]. We built a controlled RAG application into which one fault of each class D1–D5 can be injected independently, plus a *clean* configuration and a *guarded-but-owner-less* configuration—a derived store keyed without an owner component but protected by a correct runtime ownership check, which is a deliberate false-positive trap for static analysis. We ran three detectors: CANARY-ECHO (ours); a *route-level IDOR scanner* that substitutes foreign resource ids on direct-object routes and flags a non-denied response, as BOLA fuzzers do [21], [19]; and a *static key-analysis* baseline that flags any derived store whose key lacks an owner component.

Table V reports the canonical single-fault result. To answer the obvious objection that a single injection per class yields no estimate of stability, we additionally ran a **multi-seed robustness study**: for each configuration we executed $N{=}1000$ *independent randomized trials*, each drawing a fresh realization of the fault—different canary tokens, different document content (section count, marker positions, vocabulary), a different benign-document realization, and a randomized worker count (2–4)—and ran the same three detectors on it. This yields 5000 class-trials per detector; Table VI reports per-class detection rate and a Wilson 95% confidence interval on each detector's aggregate recall.

TABLE VI
RQ3 MULTI-SEED ROBUSTNESS: $N{=}1000$ INDEPENDENT RANDOMIZED TRIALS PER CONFIGURATION (5000 CLASS-TRIALS PER DETECTOR). PER-CLASS DETECTION RATE (%) AND AGGREGATE RECALL WITH WILSON 95% CI. "FP_g" = FALSE-POSITIVE RATE ON THE GUARDED-BUT-OWNER-LESS STORE.

Detector	D1	D2	D3	D4	D5	Recall (95% CI)	FP_g
CANARY-ECHO	99	99	98	100	100	**99.3** [99.0,99.5]	0
IDOR	0	0	0	0	100	20.0 [18.9,21.1]	0
Static	100	100	100	100	0	80.0 [78.9,81.1]	100

Per-class values are detection rate (%); recall and CI in %; FP_g in %.

B. Finding

CANARY-ECHO detects all five classes with no false positive in the canonical run, and the robustness study confirms the result is stable, not a lucky seed: aggregate recall is 99.3% (Wilson 95% CI $[99.0\%, 99.5\%]$) over 5000 randomized class-trials, with per-class detection 98–100% and a false-positive rate of 0% on both the clean and the guarded-but-owner-less app across all 1000 trials. The handful of misses are confined to D2 and D3 and trace to the finite per-trial probe budget occasionally not scheduling a query onto the stale worker index or not ranking the contaminated chunk into the top-k under an adversarial random document—a *probe-coverage* limit, not an oracle error (the content oracle never once fired on content that had not crossed tenants). The IDOR scanner is pinned at 20.0% ($[18.9\%, 21.1\%]$): it exercises the route layer, which is exactly where D1–D4 are not; this is the same blind spot we observed in Open WebUI (RQ1), where the route checks passed while the derived-layer checks failed, and in the public record (RQ2), which is entirely D5. Static key analysis is pinned at 80.0% ($[78.9\%, 81.1\%]$): it detects the four derived-layer classes but misses D5—a route-level mistake over correctly-keyed storage, invisible to a key-shape rule—and *false-positives on every one* of the 1000 guarded-but-owner-less trials, because it cannot see the runtime guard. The separation between the three detectors is therefore not a within-noise artifact: the confidence intervals do not overlap, and each baseline is blind to a region the dynamic, content-based method covers. In practice a team would run route-level BOLA testing *and* CANARY-ECHO; the point of RQ3 is that neither the route scanner nor static analysis is a substitute for observing the derived layer dynamically.

VIII. DISCUSSION

A. Isolation failure as a reliability defect

We have framed cross-tenant leakage as a security issue, but it is first a *dependability* failure, and treating it as one explains why it survives conventional assurance. A multi-tenant service makes an implicit correctness contract—each tenant's answers are computed only from that tenant's authorized data—and a derived-state leak violates that contract silently: the request succeeds, the status is 200, no exception is raised, and the latency is nominal. Three standard reliability instruments therefore cannot see it. Functional tests pass, because each tenant's own queries return correct answers; the defect is

only visible in the *relation* between two tenants' executions, which single-execution test cases do not state. Runtime health monitoring is silent, because there is no crash, error rate, or latency excursion to alarm on—the failure has no observable *event*. And software-reliability growth models, which estimate failure intensity and MTBF from an observed failure process [9], cannot estimate the intensity of a failure that produces no signal to count; a system can leak on a large fraction of cross-tenant interactions while its measured failure intensity is zero. This is the defining shape of a *silent* correctness failure, and it is precisely the class a dependable multi-tenant system most needs to exclude, because the damage—one tenant's confidential corpus surfacing in another's answer—is irreversible once it reaches a user and erodes the trust assumption on which the shared deployment rests. Our contribution, read in reliability terms, is to give this silent failure an *observable* proxy (the canary predicate) and a *differential* test case form (the two-tenant metamorphic relation), turning a property that growth models cannot measure into one a fault-injection study can quantify—which is exactly what RQ3's recall-with-confidence-interval does. The method extends reliability testing to a correctness property that the usual failure-event machinery misses, rather than merely porting a security scanner.

a) Why route-level testing is necessary but not sufficient: RQ2 shows the public record is entirely route-level; RQ1 and RQ3 show that derived-layer leaks nevertheless occur and that neither IDOR scanning nor static key analysis catches the whole picture. BOLA/IDOR testing [21], [19], [20] remains essential—D5 is real and common, and tops the OWASP API list [11]—but isolation testing for RAG must additionally observe the derived layer, which is what the echo service and the O-prompt/O-art oracles do. The two lines are complementary, not competing: a complete pre-launch check runs both, and our seeded-fault study quantifies exactly what each misses.

b) Closest baseline, made concrete: The nearest prior work is the addition of BOLA/BFLA oracles to a REST fuzzer [21]: for a resource owned by a, the fuzzer replays a's request as b and flags a violation when the response status is not a denial ($401/403$). This is the right test for D5 and we do not improve on it there. The difference is the observation point and the decision rule. Their oracle reads the *status* of the *direct-object* response; ours reads the *content* of *every* observation point, including the LLM prompt and persisted artifacts that the direct-object route never produces. Concretely, on the Open WebUI leak (RQ1) a status-BOLA oracle sees `GET /files/<id>` $\rightarrow 404$ and correctly concludes the route is sound—and stops, because it has no notion of a retrieval path that re-reads the same file by id. Our oracle additionally issues the chat request that carries the foreign id into retrieval and observes a's canary in b's prompt. The two oracles agree on D5 and diverge exactly on D1–D4, which is why RQ3 reports 20% recall for the status oracle and 100% for ours: not because the status oracle is wrong about what it tests, but because the derived layer is outside what it tests. Symmetrically, the content rule repairs a precision problem the status rule has even at the route (the AnythingLLM 200-null denials of RQ1), so the content-based decision dominates the status-based one on both recall and precision.

c) Generality and cost: The method applies to any RAG application that can point its chat and embedding models at a custom OpenAI-compatible endpoint, which covers the large majority of self-hosted systems. Adding an application costs one adapter (80–180 lines); the canary and oracle design is independent of the vector store, the embedding model, and the LLM, and the echo service runs on a CPU with no API key, so a full run of the suite is a matter of minutes and dollars-free. This cost profile matters for a reliability practice: a check cheap enough to run on every release is one that will actually be run.

d) Operational integration: Because the suite is black-box and deterministic, it fits an ordinary continuous-integration stage: spin up the application under test with its model endpoints pointed at the echo service, run the adapter and scenario battery, and fail the build on any non-empty leak set. The owner-tagged canary registry makes each failure self-describing—owner, observer, oracle, and scenario—so a regression points directly at the derived store that leaked rather than at a generic assertion failure. A team adopting the method incrementally can begin with S1, S5, and S7 (own-document sanity, the route surface, and the derived-layer reference), which together already separate "route correct, derived layer leaking" from "both correct," and add the remaining scenarios as their worker topology and caching layers warrant. The determinism is what makes this practical: a flaky isolation test that fails once in ten runs would be disabled within a sprint, whereas a substring predicate over echoed canaries either fires or does not.

B. Threats to validity

a) Construct: Deterministic echo embeddings are not the application's production model; we test the *plumbing* of isolation (which store serves which tenant), not retrieval quality, and the canary predicate does not depend on embedding fidelity. One construct limitation is concrete: applications whose pipeline uses the model to decide whether and what to retrieve (observed in Khoj) cannot be driven end-to-end by a trivial echo; for those we confirm at the retrieval/authorization layer with the application's own code, and a retrieval-planning-aware mock LLM is future work.

b) Internal: Every dynamic finding has a positive and a negative control (a vulnerable vs. a fixed version, or a leaking vs. a guarded configuration), and the S1 own-document check guards against crediting a "clean" verdict to an adapter that never exercised retrieval. Test subjects are always regular users; the suite bootstraps a separate administrator so that by-design administrative access is not miscounted. Route-level denial is judged by content, not status code, removing a class of false positives demonstrated on AnythingLLM (RQ1).

c) External: Six applications are not the ecosystem, and two were confirmed only at the source level; broadening the dynamic set is the main threat to external validity. The seeded-fault study, by contrast, is now backed by 1000 randomized trials per class with reported confidence intervals

(Section VII), so its effectiveness claim is not a single-seed artifact; its remaining limitation is that the seeded application is a controlled model of the five server-side mechanisms rather than production code, so its absolute recall need not transfer verbatim to a specific system, even though the *separation* between detectors is robust. The historical corpus is one CVE per application; a census of *all* isolation CVEs per application would sharpen the "derived layer is absent" claim. The inter-coder study uses an LLM as the second annotator; human replication is the stronger form, and the materials are released for it.

d) Ethics and responsible disclosure: All tests ran against self-deployed, isolated instances only; no third-party or production service was touched, and the released artifacts drive only local deployments. Every real-world finding is either an already-public CVE or a leak fixed in the current release, so we introduce no new undisclosed vulnerability; for the one live, unfixed CVE (Khoj) we rely on the vendor's existing public advisory and open fix pull request, and add no exploitation detail beyond what is needed to substantiate the classification.

IX. Related Work

a) RAG and LLM security: Work on RAG security has centered on attacks against the model and its data: extraction of training or context data [12], [13], direct and indirect prompt injection through retrieved content [14], [30], and knowledge-base poisoning [15]; the OWASP LLM Top 10 names vector/embedding weaknesses as a category [10]. These study how an adversary manipulates *what the model knows or says*. We study whether a deployed application's *derived state* already leaks one tenant's documents to another, independent of any model attack—a question about the engineering of isolation, not about the model.

b) Authorization testing for web and REST APIs: There is a mature line on testing authorization in REST APIs. General REST fuzzers and test generators—RESTler [25], EvoMaster [31], RestTestGen [32], RESTest [33], Morest [34], MINER [35], NAUTILUS [36], ARAT-RL [37], VOAPI2 [38], and tools examined in recent surveys, comparisons, and coverage studies [26], [27], [39]—maximize schema and code coverage and detect faults chiefly via server-error (5xx) status codes. Security-focused extensions check authorization specifically: Atlidakis et al. check security properties including resource-after-delete and cross-namespace access [18]; Corradini et al. detect mass-assignment vulnerabilities [19]; Du et al. target vulnerability-oriented testing [38]; Santos Filho et al. detect broken object-level authorization by transforming OpenAPI specifications to colored Petri nets [20]; and, most closely to us, Arcuri et al. add automated oracles for access-policy violations (BOLA/BFLA) to a REST fuzzer, inferring violations from $401/403$ responses [21]. All of these operate at the *route*: they decide authorization from the direct-object response and its status code. Our contribution is orthogonal and complementary—we observe the *derived state* beneath a correctly authorized route, and we show (RQ1, RQ3) that a status-based route oracle both misses derived-layer leaks and misclassifies non-standard denials. Policy-coverage and access-control-policy testing [40], [41] similarly reason about the specified policy, not about derived indexes that no policy mentions.

c) Metamorphic, differential, and oracle-free testing: Our oracle is a metamorphic relation—"tenant a's content must not appear for tenant b" [22], [23]—instantiated concretely by canaries and evaluated *differentially* across tenants [42]. Metamorphic security testing has been applied to web systems [43]; we specialize it to RAG tenant isolation, where the generative component makes a direct correctness oracle impractical [24] and a canary-based relation sidesteps it. Fuzzing more broadly [44] shares our black-box, input-generation stance but relies on crash or coverage oracles that do not capture cross-tenant content flow, which is a functional-correctness property rather than a memory-safety one.

d) Multi-tenant isolation: Cross-tenant information leakage on shared infrastructure is a long-standing concern in cloud computing [16], [17]; those leaks exploit shared hardware beneath the application, whereas ours arise *inside* a single application's retrieval indexes and caches. The hazard—sharing state keyed for efficiency rather than for ownership—is the same; the layer and the remedy differ, and the RAG setting adds a new twist: the leaking channel is the application's *intended* retrieval path, not an unintended side channel.

X. Conclusion

Isolation in multi-tenant RAG is enforced where it is easy to see, at the API route, and fails where it is not, in the derived state—vector namespaces, lexical indexes, caches, and worker indexes—that assembles the answer. CANARY-ECHO differential testing makes that layer observable with owner-identifying canaries and a deterministic, GPU-free echo service, and decides leaks with four content-based oracles over a metamorphic cross-tenant relation. Across six open-source applications it finds two independent real leaks beneath correct routes with no false alarms; across ten public CVEs it shows that the derived layer is absent from the record while route-level bugs fill it; and on seeded faults it detects every server-side class that route-level and static baselines miss. The derived layer is where multi-tenant RAG leaks—and, with the right canaries and oracles, it is testable before launch.

Several directions would strengthen the method and the evidence. A retrieval-planning-aware mock model—one that satisfies the structured intermediate decisions a pipeline like Khoj's makes, while still echoing canaries deterministically—would let the echo technique drive end-to-end answers in applications that currently require source-level confirmation. On the empirical side, a human second annotator would replace the LLM-based inter-coder measurement, a census of *all* isolation CVEs per application (rather than one representative each) would test the "derived layer is absent from the public record" claim at larger scale, and multiple independent seeds per fault class would put confidence intervals on the RQ3 recall figures. Finally, partial synthesis of adapters from an application's OpenAPI description would lower the per-application cost

further and make derived-state isolation testing a routine part of every RAG release.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[2] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[3] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[4] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[5] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[7] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. on Research and Development in Information Retrieval*, 2009, pp. 758–759.

[8] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. 16th Conf. of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021.

[9] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[10] OWASP Foundation, "OWASP top 10 for large language model applications (2025): LLM08 vector and embedding weaknesses," OWASP GenAI Security Project, 2025.

[11] ——, "OWASP API security top 10 (2023): API1 broken object level authorization," OWASP API Security Project, 2023.

[12] N. Carlini, F. Tramer, E. Wallace, M. Jagielski, A. Herbert-Voss, K. Lee, A. Roberts, T. Brown, D. Song, Ú. Erlingsson, A. Oprea, and C. Raffel, "Extracting training data from large language models," in *Proc. 30th USENIX Security Symposium*, 2021, pp. 2633–2650.

[13] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, and J. Tang, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[15] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," in *Proc. 34th USENIX Security Symposium*, 2025.

[16] T. Ristenpart, E. Tromer, H. Shacham, and S. Savage, "Hey, you, get off of my cloud: Exploring information leakage in third-party compute clouds," in *Proc. 16th ACM Conf. on Computer and Communications Security (CCS)*, 2009, pp. 199–212.

[17] Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-VM side channels and their use to extract private keys," in *Proc. 2012 ACM Conf. on Computer and Communications Security (CCS)*, 2012, pp. 305–316.

[18] V. Atlidakis, P. Godefroid, and M. Polishchuk, "Checking security properties of cloud service REST APIs," in *Proc. IEEE 13th Int. Conf. on Software Testing, Validation and Verification (ICST)*, 2020, pp. 387–397.

[19] D. Corradini, M. Montolli, M. Pasqua, and M. Ceccato, "Automated black-box testing of mass assignment vulnerabilities in RESTful APIs," in *Proc. IEEE/ACM 45th Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 2553–2564.

[20] A. Santos Filho, I. Rodríguez, and E. L. Feitosa, "Automated broken object-level authorization attack detection in REST APIs through OpenAPI to colored Petri nets transformation," *International Journal of Information Security*, vol. 24, no. 2, 2025.

[21] A. Arcuri, O. Sahin, and M. Zhang, "Fuzzing for detecting access policy violations in REST APIs," in *Proc. IEEE 36th Int. Symp. on Software Reliability Engineering (ISSRE)*, 2025, pp. 130–141.

[22] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–27, 2018.

[23] S. Segura, G. Fraser, A. B. Sanchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[24] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[25] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API fuzzing," in *Proc. IEEE/ACM 41st Int. Conf. on Software Engineering (ICSE)*, 2019, pp. 748–758.

[26] A. Golmohammadi, M. Zhang, and A. Arcuri, "Testing RESTful APIs: A survey," *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 1, pp. 1–41, 2023.

[27] M. Zhang and A. Arcuri, "Open problems in fuzzing RESTful APIs: A comparison of tools," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–45, 2023.

[28] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[29] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[30] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *Proc. 33rd USENIX Security Symposium*, 2024.

[31] A. Arcuri, "RESTful API automated test case generation with EvoMaster," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 1, pp. 1–37, 2019.

[32] E. Viglianisi, M. Dallago, and M. Ceccato, "RestTestGen: Automated black-box testing of RESTful APIs," in *Proc. IEEE 13th Int. Conf. on Software Testing, Validation and Verification (ICST)*, 2020.

[33] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "RESTest: Automated black-box testing of RESTful web APIs," in *Proc. 30th ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2021, pp. 682–685.

[34] Y. Liu, Y. Li, G. Deng, Y. Liu, R. Wan, R. Wu, D. Ji, S. Xu, and M. Bao, "Morest: Model-based RESTful API testing with execution feedback," in *Proc. IEEE/ACM 44th Int. Conf. on Software Engineering (ICSE)*, 2022, pp. 1406–1417.

[35] C. Lyu, J. Xu, S. Ji, X. Zhang, Q. Wang, B. Zhao, G. Pan, W. Cao, P. Chen, and R. Beyah, "MINER: A hybrid data-driven approach for REST API fuzzing," in *Proc. 32nd USENIX Security Symposium*, 2023, pp. 4517–4534.

[36] G. Deng, Z. Zhang, Y. Li, Y. Liu, T. Zhang, Y. Liu, G. Yu, and D. Wang, "NAUTILUS: Automated RESTful API vulnerability detection," in *Proc. 32nd USENIX Security Symposium*, 2023, pp. 5593–5609.

[37] M. Kim, S. Sinha, and A. Orso, "Adaptive REST API testing with reinforcement learning," in *Proc. IEEE/ACM 38th Int. Conf. on Automated Software Engineering (ASE)*, 2023, pp. 446–458.

[38] W. Du, J. Li, Y. Wang, L. Chen, R. Zhao, J. Zhu, Z. Han, Y. Wang, and H. Zu, "Vulnerability-oriented testing for RESTful APIs," in *Proc. 33rd USENIX Security Symposium*, 2024, pp. 739–755.

[39] M. Kim, Q. Xin, S. Sinha, and A. Orso, "Automated test generation for REST APIs: No time to rest yet," in *Proc. 31st ACM SIGSOFT Int. Symp. on Software Testing and Analysis (ISSTA)*, 2022, pp. 289–301.

[40] E. Martin, T. Xie, and T. Yu, "Defining and measuring policy coverage in testing access control policies," in *Proc. 8th Int. Conf. on Information and Communications Security (ICICS)*, 2006, pp. 139–158.

[41] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, "Verification and change-impact analysis of access-control policies," in *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 196–205.

[42] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100–107, 1998.

[43] P. X. Mai, F. Pastore, A. Goknil, and L. Briand, "Metamorphic security testing for web systems," in *Proc. IEEE 13th Int. Conf. on Software Testing, Validation and Verification (ICST)*, 2020, pp. 186–197.

[44] V. J. M. Manès, H. Han, C. Han, S. K. Cha, M. Egele, E. J. Schwartz, and M. Woo, "The art, science, and engineering of fuzzing: A survey," *IEEE Transactions on Software Engineering*, vol. 47, no. 11, pp. 2312–2331, 2021.